

Lecture 01 From answers to certificates

Prerequisites: Python, inequalities and elementary calculus.

Goal: turn a plausible numerical answer into a precise claim with checkable evidence.

1. What does a computed number tell us?

0–12 minutes: predictions, discussion, and vocabulary

A program returns a finite object: a bit pattern, a printed decimal, a list of points. The mathematical question usually concerns an exact real number or every point in an infinite set. Validated computing supplies a justified connection between those objects. Before discussing precision, ask: *what exact statement are we trying to establish?*

Predict these results individually, then compare explanations with a neighbour.

```
print(0.1 + 0.2 == 0.3)
large = 1e16
print((large + 1.0) - large)
x = 1.4142
print(x*x - 2.0)
```

On the binary64 Python environment used in this course, the first result is `False` and the second is `0.0`. The third is close to zero. The first two expose representation or rounding; the third raises a different question: how does a small function value constrain the distance to a root?

Four levels of statement. An *approximation* proposes a value, such as 1.4142. An *error estimate* predicts its accuracy under stated assumptions, which may still need checking. An *enclosure* is a set proved to contain the intended exact answer. A *certificate* combines a precise claim with finite evidence and the mathematical argument explaining why the evidence suffices.

For example, “two runs print the same digits” is evidence about two computations. “The positive root of $x^2 - 2$ belongs to $[1.4142, 1.4143]$ ” is a claim about an exact real number. Agreement can suggest the claim, but cannot replace its proof: two runs may share the same mistake or discard the same information.

Discuss: which missing assumption matters?

A plot crosses the horizontal axis; two endpoint evaluations have opposite signs; an iteration has stopped changing; two precisions agree. Which of these establishes a root? For each, name one additional condition or piece of evidence you would need.

Working rule. Write the claim before writing the checker. “Exactly one root in this interval” requires existence *and* uniqueness. “No root here” requires exclusion on the entire interval. “I could not decide” is a useful outcome, not a licence to replace a missing proof with a favourable plot.

Two jobs for a numerical program. An exploratory calculation may locate a promising interval quickly, using plots, ordinary floating-point arithmetic, or a heuristic search. A validation then checks a sufficient mathematical condition on that interval. The search need not be trusted if the final checker independently verifies all required evidence. This separation lets us retain fast exploratory methods while making the final conclusion depend on a much smaller argument.

2. Identify the input and the error

12–25 minutes: exact arithmetic and a short input audit

There are several places for a numerical answer to depart from the intended problem. Keeping them separate helps us choose the right remedy.

Source	Example and question to ask
Input representation	Does the binary float labelled 0.1 equal the rational 1/10?
Arithmetic rounding	Was an increment discarded during an addition?
Truncation	How large is the omitted tail of an infinite sum?
Discretization	What can happen between the sampled grid points?
Input uncertainty	Is a measurement an exact value or a whole admissible range?
Model discrepancy	Does the mathematical model describe the physical system adequately?

A proof about a mathematical model does not automatically bound its discrepancy from the physical world. Increasing arithmetic precision addresses neither an unmodelled force nor an uncertain sensor reading.

Exact audit of a familiar decimal. The stored binary64 value of 0.1 is

$$q = \frac{3602879701896397}{36028797018963968}, \quad q - \frac{1}{10} = \frac{1}{180143985094819840} > 0.$$

Both identities concern rational numbers, so Python’s integer and rational arithmetic can check them without using an approximate decimal reference.

```
from fractions import Fraction
intended = Fraction(1, 10)
stored = Fraction.from_float(0.1)
assert stored - intended == Fraction(1, 180143985094819840)
assert Fraction("0.1") == intended
assert Fraction(0.1) == stored
```

The last line does not recover the user’s intended decimal: the float conversion has already happened. Also distinguish 1/10 from a measurement reported as 0.1 ± 0.001 . The latter means an input set, even if its endpoints are represented exactly.

Absolute and relative error. If the intended answer is y and the computed answer is $\hat{y}$, absolute error is $|\hat{y} - y|$. Relative error is $|\hat{y} - y|/|y|$, defined only when $y \neq 0$. Neither can normally be computed exactly unless we already know enough about y . A proved upper bound e gives

$$|\hat{y} - y| \leq e \iff y \in [\hat{y} - e, \hat{y} + e].$$

The endpoints must themselves be computed exactly or rounded outwards.

Pair question

An instrument reports a length as 2.000 ± 0.005 metres. Would 100-digit arithmetic make the length known to 100 digits? Which error source would remain?

Choose an error scale for the question. An absolute error of one unit is small relatively when the true answer is 10^9 , but enormous when it is 10^{-9} . Near a zero, relative error can be misleading or undefined, while an absolute tolerance remains meaningful. A sensible report states both the error metric and its units. “Accurate to six digits” is incomplete unless the intended meaning, such as a relative bound or correct decimal rounding, is specified.

3. Build a complete root certificate

25–45 minutes: board proof followed by an exact checker

Let $f(x) = x^2 - 2$. We want to prove that there is exactly one positive root in

$$I = [a, b], \quad a = \frac{14142}{10000}, \quad b = \frac{14143}{10000}.$$

A complete argument has three parts. First, a polynomial is continuous on I . Second, exact integer arithmetic gives

$$14142^2 = 199996164 < 200000000 < 200024449 = 14143^2,$$

so $f(a) < 0 < f(b)$. The intermediate value theorem therefore gives a root in (a, b) . Third, $f'(x) = 2x > 0$ on I , so f is strictly increasing and the root there is unique. The sign checks provide evidence; continuity and monotonicity explain its force.

```
from fractions import Fraction

a = Fraction(14142, 10000)
b = Fraction(14143, 10000)
assert 0 < a < b
assert a*a < 2 < b*b
midpoint = (a + b) / 2
radius = (b - a) / 2
print(midpoint, radius) # 5657/4000 and 1/20000
```

The midpoint is 1.41425 and the radius is 0.00005. Consequently the exact positive root r satisfies $|r - 1.41425| \leq 0.00005$. This enclosure is already a validated result, although no interval-arithmetic library was needed.

Improve the certificate without knowing the root. Put $m = (a + b)/2$. If $m^2 < 2$, replace a by m ; if $m^2 > 2$, replace b by m . If $m^2 = 2$, the midpoint is an exact root and we can stop. The sign bracket is preserved, and each nonterminal step halves its width. After n steps the width is $(b_0 - a_0)2^{-n}$. The midpoint error is at most half that width. Thus the desired accuracy determines a finite number of steps in advance.

Board activity: refine and interpret

Perform the first two bisections using fractions. Keep a table of the endpoints, midpoint sign, and interval width. Which statement is the loop invariant? Does the original bracket alone certify that rounding the root to four decimal places gives 1.4142? Locate the rounding threshold before answering.

Certificate record. Save the function, exact endpoints, endpoint signs, existence theorem, uniqueness argument, and final width. A second reader should be able to check this record without trusting the code that guessed the bracket. Our reasoning trusts elementary real analysis and the exact-arithmetic implementation; it is not a formal verification of the Python interpreter.

When may we print certified digits? To certify rounding to p decimal places, it is sufficient for the entire enclosure to lie strictly inside one rounding bin $((k - \frac{1}{2})10^{-p}, (k + \frac{1}{2})10^{-p})$ for an integer k . Then every possible exact answer rounds to $k10^{-p}$. A small interval that straddles a rounding boundary does not decide the rounded answer. Width alone therefore does not determine which digits are certified.

4. Why common stopping tests are incomplete

45–60 minutes: two counterexamples and two useful bounds

Residual versus error. A residual measures how closely an equation is satisfied. For $h(x) = 10^{-12}(x - 1)$, the residual at $x = 0$ has magnitude 10^{-12} , but the distance to the unique root is 1. Multiplying an equation by a small constant makes residuals small without moving any root.

A theorem can connect a residual to an error. Suppose r is a known root, f is continuous on the segment between r and x and differentiable inside it, and $|f'(t)| \geq m > 0$ throughout that segment. The mean value theorem gives

$$f(x) - f(r) = f'(\xi)(x - r), \quad |x - r| \leq \frac{|f(x)|}{m}.$$

This is conditional on existence of r and the derivative bound. Neither condition follows merely from observing a small residual. If the residual is computed with rounding, we also need a justified upper bound on its exact magnitude.

For the root certified on page 3, $r, x \in [1, 2]$ implies $f'(t) \geq 2$. With $x = 1.4142$ interpreted as an exact decimal, $|f(x)| = 0.00003836$, so $|x - r| \leq 0.00001918$. This improves the one-sided endpoint distance bound obtained from the original bracket. The root-existence proof was an essential earlier step.

A positive grid does not prove positivity. The function $g(x) = (x - 1/3)^2$ vanishes at $1/3$, yet is strictly positive at each point $0, 0.1, \dots, 1$. Even exact evaluation at every point of that grid misses the zero. The issue is coverage between points, not floating-point accuracy.

Here is one way to bridge that gap. Assume $|g'(x)| \leq L$ on $[a, b]$, and let a uniform grid include both endpoints with spacing h . Every point is within $h/2$ of a grid point x_j . The mean value theorem yields

$$g(x) \geq g(x_j) - L|x - x_j| \geq \min_j g(x_j) - Lh/2.$$

If the final lower bound is positive, then g is positive everywhere on $[a, b]$. Use certified lower bounds for the grid values if their evaluations are rounded.

Apply the theorem, not just the picture

For $g(x) = x^2 + 1$ on $[-1, 1]$, use $L = 2$ and the grid $-1, -1/2, 0, 1/2, 1$. What global lower bound follows? Is it sharp? If a bound is not positive, have we proved that a zero exists?

Do not omit continuity. The expression $f(x) = 1/x$ has opposite signs at -1 and 1 , but no real zero. It is not defined at the interior point zero, so the intermediate value theorem does not apply on $[-1, 1]$. Sampling more points without checking the domain does not repair that argument. Root validation must check the hypotheses of the theorem as carefully as the numerical inequalities.

5. Bound what was never computed

60–75 minutes: a tail estimate and a combined error budget

An exact finite computation can still answer the wrong question if it omits an uncontrolled infinite tail. Consider

$$S = \sum_{k=1}^{\infty} \frac{1}{k^2}, \quad S_N = \sum_{k=1}^N \frac{1}{k^2}, \quad R_N = S - S_N.$$

We do not need a closed form for S to enclose it. Since $x \mapsto x^{-2}$ is positive and decreasing, comparison with rectangles under and over the curve gives

$$\int_{N+1}^{\infty} \frac{dx}{x^2} \leq R_N \leq \int_N^{\infty} \frac{dx}{x^2}.$$

For the lower inequality, compare $1/k^2$ with the integral over $[k, k+1]$; for the upper one, compare it with the integral over $[k-1, k]$. Summing from $k = N+1$ and evaluating the integrals proves

$$S_N + \frac{1}{N+1} \leq S \leq S_N + \frac{1}{N}.$$

Every endpoint is rational when S_N is accumulated exactly. Notice that the *enclosure width* is $1/[N(N+1)]$, while the omitted tail itself is of order $1/N$. Bounding the tail from both sides is more informative than discarding it.

```
from fractions import Fraction

N = 10
partial = Fraction(0)
for k in range(1, N + 1):
    partial += Fraction(1, k*k)
lower = partial + Fraction(1, N + 1)
upper = partial + Fraction(1, N)
assert upper - lower == Fraction(1, N*(N + 1))
print(lower, upper)
```

A float sum $\hat{S}_N$ would need a separate arithmetic bound. If $|\hat{S}_N - S_N| \leq \rho$, then a valid enclosure is

$$\left[\hat{S}_N - \rho + \frac{1}{N+1}, \hat{S}_N + \rho + \frac{1}{N} \right],$$

provided its endpoints are formed with exact arithmetic or outward rounding. Truncation control does not secretly include summation error.

Explain the error budget

One student increases N but keeps an unreliable summation method. Another computes S_{10} to 200 digits but ignores the tail. What remains unjustified in each case? How large must N be for the exact enclosure width to be below 10^{-4} ?

Why not use fractions for everything? Exact rationals give transparent reference calculations, but their numerators and denominators can grow large. They also do not directly represent arbitrary square roots or transcendental values. Later lectures replace exact intermediate values with rigorous enclosures whose endpoints have controlled finite precision. This trades exact equality for a proved inclusion, retaining the mathematical guarantee while making more complex computations practical.

6. Exercises and a certificate audit

75–90 minutes: choose two problems in pairs; finish the others after class

For each answer, distinguish observations, hypotheses, and conclusions. A useful submission includes the intended exact inputs and explains why the checks imply the stated claim. Code alone is not an existence or uniqueness argument.

1. **A new root certificate.** Prove that the positive square root of 3 lies in $[1732/1000, 1733/1000]$. State the theorem for existence and the argument for uniqueness. Give a midpoint and certified error bound. How many bisections make the width at most 10^{-6} ? Use integer comparisons, not a call to `math.sqrt`, as the evidence.
2. **Same endpoint signs.** On $[-1, 1]$, give three continuous functions that are positive at both endpoints and have, respectively, no root, exactly one root, and two distinct roots in the interior. What stronger information would allow you to exclude a root? Explain why making a plot denser does not supply it.
3. **Repair a numerical report.** A program states: “The smallest of 101 sampled values is 0.03, so $g(x) > 0$ on $[0, 1]$.” Assume the uniform grid includes both endpoints and each value is exact. If $|g'| \leq 4$, repair the argument and give a certified lower bound. Repeat with $|g'| \leq 10$ and report honestly what the bound does, and does not, decide.
4. **Combine two bounds.** Suppose $N = 100$, a finite-sum approximation is $\hat{S}_N$, and $|\hat{S}_N - S_N| \leq 10^{-8}$. Write a rigorous enclosure for the infinite sum of page 5. What is its width? Identify the arithmetic that still needs care when representing its endpoints on a machine.

Audit another pair’s solution

Can you find the exact claim, domain, and input interpretation? Which facts are checked by arithmetic and which are supplied by a theorem? Are all theorem hypotheses stated? What would the program report if one check failed?

Selected checks, after attempting the problems. For Exercise 1, $1732^2 = 2999824$ and $1733^2 = 3003289$; the midpoint is 1.7325 and the radius 0.0005. Ten bisections suffice. For Exercise 3, the covering radius is $1/200$, giving lower bounds 0.01 and -0.02 . The latter is inconclusive, not evidence of a negative value. For Exercise 4 the width is $1/(100 \cdot 101) + 2 \cdot 10^{-8}$.

Exit question. Describe a small computation that you could audit without rerunning the search that produced its answer. What would you save as its certificate?

Reading and notebook. Companion: `notebooks/01_intro.ipynb`; Lab 01: numerical crime scene. Tucker, *Validated Numerics* (2011), Chapter 1, especially §1.2 and §1.6; Example 1.6.3 motivates combining a finite sum with a tail bound. The arguments and exercises above are expanded course material.